

Relatório Técnico

Practical Assignment

Trabalho Prático

Computação Gráfica



Universidade do Minho

Universidade do Minho

Escola de Engenharia, Braga, Portugal
Licenciatura em Engenharia Informática

Ano Letivo 2025/2026

Grupo 25

Carlos Silva; Francisco Veloso; Nuno Soares
a106792@uminho.pt
a106882@uminho.pt
a107366@uminho.pt

Conteúdo

1	Introdução	4
2	Fase 1: Primitivas e Motor de Visualização	5
2.1	Fluxo de Dados	5
2.2	Generator — Geração de Primitivas	5
2.2.1	Plano (Plane)	5
2.2.2	Algoritmo	5
2.2.3	Caixa (Box)	6
2.2.4	Algoritmo	6
2.2.5	Esfera (Sphere)	6
2.2.6	Algoritmo	7
2.2.7	Cone	7
2.2.8	Geometria do Cone	8
2.2.9	Algoritmo	8
2.3	Engine — Motor Gráfico	8
2.3.1	Formato do Ficheiro .3d	8
2.3.2	Leitura da Configuração XML	9
2.3.3	Pipeline de Renderização	9
2.3.4	Callback de Reshape	9
2.3.5	Callback de Display	9
2.3.6	Sistema de Câmera	10
2.3.7	Câmera Fixa (XML)	10
2.3.8	Câmera Orbital (Explorer)	10
2.3.9	Modos de Renderização	10
2.3.10	Controlos Adicionais	10
2.4	Resultados obtidos	11
2.5	Conclusão	13
3	Fase 2: Transformações Geométricas e Hierarquia de Grupos	14
3.1	Objetivo da Fase 2	14
3.2	Arquitetura Introduzida	14
3.2.1	Estrutura Transform	14
3.2.2	Estrutura Group	14
3.3	Parsing XML Recursivo	14
3.3.1	Leitura das Transformações	14
3.3.2	Leitura dos Modelos	14
3.3.3	Leitura de Subgrupos	15
3.4	Renderização Hierárquica	15
3.4.1	Aplicação das Transformações	15
3.5	Validação com Testes da Fase 2	15
3.6	Resultados Obtidos	16
3.7	Conclusão da Fase 2	17
4	Fase 3: Curvas de Catmull-Rom, Superfícies de Bezier e VBOs	18
4.1	Objetivo da Fase 3	18
4.2	Geração das Superfícies de Bezier	18
4.2.1	Estratégia de Tesselação	18
4.2.2	Formato do Ficheiro Gerado	18
4.3	Curvas de Catmull-Rom no Engine	18
4.3.1	Cálculo da Curva	18
4.3.2	Alinhamento com a Trajetória	18
4.3.3	Visualização da Curva	19
4.4	VBOs	19

4.5	Parsing XML da Fase 3	19
4.6	Validação com os Testes da Fase 3	19
4.7	Resultados Obtidos	20
4.8	Conclusão da Fase 3	20
5	Fase 4: Normais, Iluminação, Materiais e Texturas	22
5.1	Objetivo da Fase 4	22
5.2	Novo Formato dos Ficheiros .3d	22
5.3	Normais e Coordenadas de Textura no Generator	22
5.3.1	Plano	22
5.3.2	Caixa	23
5.3.3	Esfera	23
5.3.4	Cone	23
5.3.5	Superfícies de Bezier	23
5.4	Carregamento no Engine e Uso de VBOs	23
5.5	Materiais no XML	24
5.6	Luzes	24
5.7	Texturas com DevIL	24
5.8	Validação com os Testes da Fase 4	25
5.9	Resultados Obtidos	25
5.10	Conclusão da Fase 4	27
6	Conclusões	28

1 Introdução

O presente relatório descreve o desenvolvimento do Trabalho Prático da unidade curricular de Computação Gráfica ao longo das suas quatro fases. O objetivo principal consistiu na construção gradual de um motor gráfico 3D, recorrendo à biblioteca OpenGL/GLUT, e na implementação de um sistema modular composto por duas aplicações distintas: um *generator*, responsável pela criação das primitivas geométricas, e um *engine*, encarregado da leitura de ficheiros de configuração e da renderização da cena.

Ao longo do trabalho, implementámos as várias primitivas tridimensionais, transformações hierárquicas, animações por curvas de Catmull-Rom, superfícies de Bezier, VBOs, iluminação, materiais e texturas. A estrutura do relatório segue a evolução do projeto, descrevendo em cada fase os objetivos, as decisões de implementação e a validação realizada com os testes fornecidos.

2 Fase 1: Primitivas e Motor de Visualização

O objetivo da Fase 1 é construir a base do motor 3D que será desenvolvido ao longo das quatro fases do trabalho. Esta fase foca-se em dois componentes fundamentais:

1. **Generator** — aplicação independente que gera ficheiros `.3d` contendo os vértices das primitivas geométricas.
2. **Engine** — aplicação que lê um ficheiro XML de configuração (câmara, janela, modelos) e renderiza a cena com OpenGL/GLUT.

A separação em dois programas distintos permite que a computação pesada da geometria seja feita apenas uma vez.

2.1 Fluxo de Dados

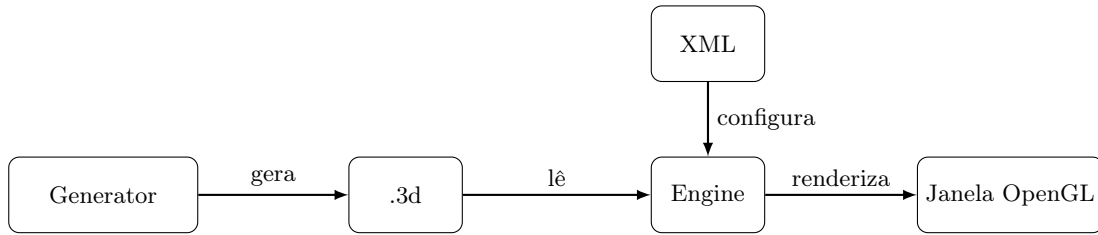


Figura 1: Fluxo de dados entre os componentes.

2.2 Generator — Geração de Primitivas

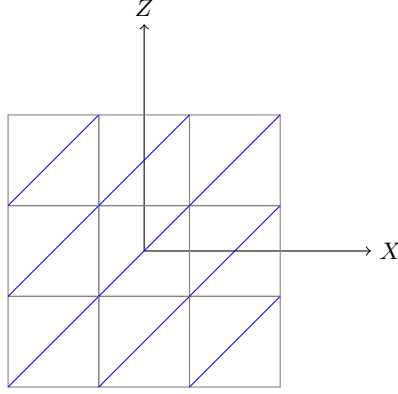
A abordagem adotada no *generator* baseia-se em superfícies paramétricas. Uma decisão fundamental foi o uso do *Counter-Clockwise Winding Order* para todos os triângulos, garantindo que as faces externas sejam corretamente renderizadas pelo OpenGL com *Back-face Culling* ativo. Ao definir os vértices nesta ordem, a face frontal de cada polígono é determinada pela regra da mão direita, permitindo que o *engine* descarte a renderização de faces internas ou não visíveis.

2.2.1 Plano (Plane)

O plano é gerado no plano XZ ($y = 0$), centrado na origem, com um dado comprimento L e número de subdivisões D .

2.2.2 Algoritmo

1. Calcular o tamanho de cada subdivisão: $s = L/D$
2. Calcular o offset de centralização: $h = L/2$
3. Para cada célula (i, j) da grelha $D \times D$:
 - Calcular os 4 cantos: $(x_1, z_1) = (-h + i \cdot s, -h + j \cdot s)$ e $(x_2, z_2) = (-h + (i+1) \cdot s, -h + (j+1) \cdot s)$
 - Emitir triângulo 1: $(x_1, 0, z_1), (x_1, 0, z_2), (x_2, 0, z_2)$
 - Emitir triângulo 2: $(x_1, 0, z_1), (x_2, 0, z_2), (x_2, 0, z_1)$



Plano com 3 divisões (vista de cima)

Figura 2: Subdivisão do plano. A diagonal azul indica a divisão de cada quad em dois triângulos.

2.2.3 Caixa (Box)

A caixa é centrada na origem com um dado tamanho S e número de divisões D por aresta. Cada uma das 6 faces é uma grelha de $D \times D$ quads, cada quad dividido em 2 triângulos.

2.2.4 Algoritmo

1. Calcular: $h = S/2$ (metade do tamanho), $s = S/D$ (passo da subdivisão)
2. Para cada uma das 6 faces (frente, trás, direita, esquerda, cima, baixo):
 - Para cada célula (i, j) da grelha $D \times D$:
 - Calcular os 4 cantos do quad na face
 - Emitir 2 triângulos com *winding* CCW (visto de fora)

Decisão de projeto: A normal de cada face aponta para fora da caixa. Os triângulos são emitidos com ordem CCW quando vistos do exterior, garantindo compatibilidade com *back-face culling*.

O número total de vértices para a caixa é:

$$V_{\text{box}} = 6 \times D^2 \times 2 \times 3 = 36 \cdot D^2$$

2.2.5 Esfera (Sphere)

A esfera é gerada a partir da conversão de coordenadas esféricas para coordenadas cartesianas. No código, adotamos a convenção em que o ângulo θ representa a componente polar e o ângulo ϕ a componente azimutal.

Equações Paramétricas:

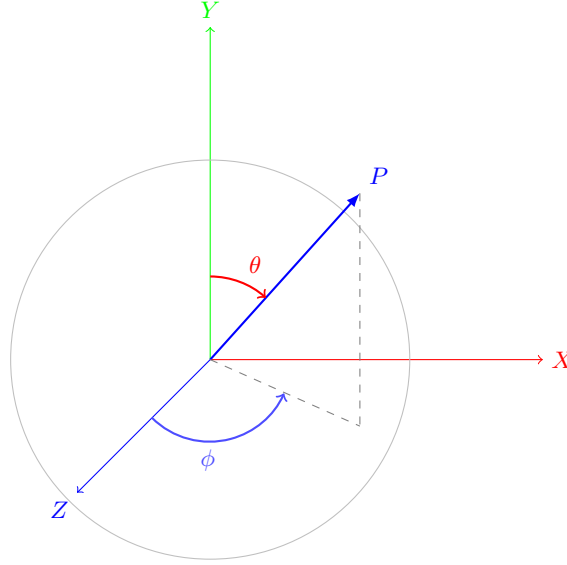
$$\begin{cases} x = r \cdot \sin(\theta) \cdot \sin(\phi) \\ y = r \cdot \cos(\theta) \\ z = r \cdot \sin(\theta) \cdot \cos(\phi) \end{cases} \quad (1)$$

A utilização direta de $\cos(\theta)$ no eixo Y mostra que este é o eixo vertical da esfera, ficando assim alinhado com o sistema de coordenadas padrão do OpenGL.

Durante a implementação, tivemos o cuidado de tratar corretamente os polos da esfera. No topo ($\theta = 0$) e na base ($\theta = \pi$), todos os pontos de cada *slice* coincidem no mesmo local. Se gerássemos quadriláteros nessas zonas (ou seja, dois triângulos por divisão), iríamos obter triângulos degenerados, com área nula.

Para evitar esse problema, o algoritmo identifica as camadas correspondentes aos polos e, nessas situações, gera apenas um triângulo por *slice*, formando um *triangle fan* que converge no polo. Desta forma, reduzimos o número de vértices desnecessários, tornando o processo ligeiramente mais eficiente.

A superfície da esfera é dividida em *slices* S_l (divisões ao longo do ângulo azimutal) e *stacks* S_t (divisões ao longo do ângulo polar). Cada quadrilátero formado entre duas *stacks* e duas *slices* é dividido em dois triângulos através da diagonal que liga o vértice v_1 ao vértice v_3 .



θ : ângulo polar, ϕ : ângulo azimutal

Figura 3: Sistema de coordenadas esféricas utilizado na geração da Esfera.

2.2.6 Algoritmo

1. Para cada stack i ($0 \leq i < S_t$):
 - Calcular $\theta_1 = \pi \cdot i / S_t$ e $\theta_2 = \pi \cdot (i+1) / S_t$
2. Para cada slice j ($0 \leq j < S_l$):
 - Calcular $\phi_1 = 2\pi \cdot j / S_l$ e $\phi_2 = 2\pi \cdot (j+1) / S_l$
 - Calcular os 4 vértices: $v_1(\theta_1, \phi_1)$, $v_2(\theta_2, \phi_1)$, $v_3(\theta_2, \phi_2)$, $v_4(\theta_1, \phi_2)$
 - Se $i \neq S_t - 1$: emitir triângulo v_1, v_2, v_3 (não degenera no polo sul)
 - Se $i \neq 0$: emitir triângulo v_1, v_3, v_4 (não degenera no polo norte)

Tratamento dos polos: No polo norte ($\theta = 0$), os vértices v_1 e v_4 coincidem no ponto $(0, r, 0)$, pelo que o triângulo $v_1 v_3 v_4$ degenera e é omitido. No polo sul ($\theta = \pi$), v_2 e v_3 coincidem em $(0, -r, 0)$, pelo que o triângulo $v_1 v_2 v_3$ degenera e é omitido.

O número total de triângulos é:

$$T_{\text{sphere}} = 2 \cdot S_l \cdot (S_t - 2) + 2 \cdot S_l = 2 \cdot S_l \cdot (S_t - 1)$$

2.2.7 Cone

O cone é gerado com a base no plano XZ ($y = 0$), centrado na origem. O raio diminui linearmente com a altura, de R (na base) a 0 (no vértice).

2.2.8 Geometria do Cone

Para uma stack i ($0 \leq i < S_t$), o raio e a altura são:

$$r_i = R \left(1 - \frac{i}{S_t} \right) \quad (2)$$

$$y_i = h \cdot \frac{i}{S_t} \quad (3)$$

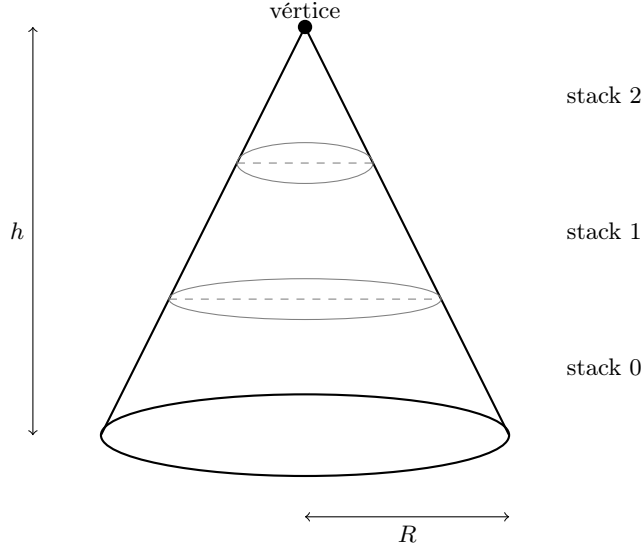


Figura 4: Estrutura do cone com 3 stacks.

2.2.9 Algoritmo

1. **Base** — leque (fan) de triângulos no plano XZ :
 - Para cada *slice* j , é emitido um triângulo com centro em $(0,0,0)$ e dois pontos consecutivos da circunferência da base (ordem CCW quando observado de baixo, com a normal orientada segundo $-Y$).
2. **Corpo** — dividido em *stacks*:
 - Para cada *stack* i e *slice* j :
 - Calculam-se os quatro vértices v_1, v_2, v_3, v_4 que definem o *quad*;
 - Emite-se o triângulo v_1, v_3, v_2 (ordem CCW quando observado do exterior);
 - Se $i \neq S_t - 1$ (ou seja, se não for a última *stack*), emite-se também o triângulo v_1, v_4, v_3 .

Na última stack, os vértices superiores convergem no vértice do cone, pelo que apenas é emitido um triângulo por slice.

2.3 Engine — Motor Gráfico

2.3.1 Formato do Ficheiro .3d

O formato de ficheiro adotado para armazenar os modelos é simples e textual:

- **Linha 1:** número total de vértices N

- **Linhas 2 a $N + 1$:** coordenadas $x y z$ de cada vértice (um por linha)

Os vértices são armazenados em grupos de 3 (formando triângulos), na ordem em que devem ser desenhados com `GL_TRIANGLES`. O *winding order* é **counter-clockwise** (CCW), que é o default do OpenGL para *front faces*.

2.3.2 Leitura da Configuração XML

Usámos a biblioteca *TinyXML-2* para interpretar o ficheiro de cena já que foi a que nos foi recomendada. A decisão de usar XML permite configurar a resolução da janela e os parâmetros da câmara sem necessidade de recompilar o código, garantindo flexibilidade na montagem de diferentes cenários.

O engine recebe como argumento um ficheiro XML que define:

- **Janela:** dimensões (largura, altura)
- **Câmara:** posição, ponto de foco (*lookAt*), vetor *up*, e parâmetros de projeção (FOV, *near*, *far*)
- **Modelos:** lista de ficheiros *.3d* a carregar

Alguns Valores por omissão são definidos para os campos opcionais (**up** = (0,1,0), **fov** = 60, **near** = 1, **far** = 1000), conforme especificado na sintaxe do file XML de referência colocado na blackboard.

2.3.3 Pipeline de Renderização

A renderização segue o pipeline clássico do OpenGL com GLUT:

1. **Inicialização:** `glutInit`, configuração do modo de display (RGBA, double buffer, depth buffer), criação da janela.
2. **Configuração OpenGL:** ativação do *depth test* e *back-face culling*.
3. **Registo de callbacks:** display, reshape, keyboard.
4. **Ciclo principal:** `glutMainLoop`.

2.3.4 Callback de Reshape

No redimensionamento da janela:

1. Ativar a matriz de projeção
2. Resetar a matriz
3. Configurar o *viewport* e aplicar a projeção perspetiva com os parâmetros do XML
4. Voltar à matriz de *modelview*

2.3.5 Callback de Display

A cada frame:

1. Limpar buffers de cor e profundidade
2. Posicionar a câmara com `gluLookAt`
3. Definir o modo de polígono (*solid/wireframe/points*)
4. Desenhando eixos de referência (com *depth test* ativo)
5. Desenhando os modelos com triângulos (modo imediato)
6. Trocar buffers (*double buffering*)

Nota: Desenhámos os eixos **antes** dos modelos, mantendo o *depth test* ativo em ambos os casos. Isto garante que a geometria dos modelos que está mais próxima da câmara se sobrepõe naturalmente aos eixos, o que respeita corretamente a profundidade.

Inicialmente, não tínhamos implementado desta forma e encontrámos alguns problemas na renderização das figuras. Embora a figura fosse estruturalmente igual à dos testes, algumas linhas que evidenciavam a profundidade não eram renderizadas corretamente.

2.3.6 Sistema de Câmera

Foram implementados dois modos de câmera:

2.3.7 Câmera Fixa (XML)

Usa diretamente os valores de posição, *lookAt* e *up* definidos no ficheiro XML.

2.3.8 Câmera Orbital (Explorer)

Implementou-se uma câmara orbital baseada em coordenadas esféricas para facilitar a inspeção dos modelos. O utilizador controla os ângulos α (rotação horizontal) e β (inclinação vertical).

Algorithm 1 Conversão para Câmera Orbital

```
1:  $pos.x \leftarrow raio \cdot \cos(\beta) \cdot \sin(\alpha)$   
2:  $pos.y \leftarrow raio \cdot \sin(\beta)$   
3:  $pos.z \leftarrow raio \cdot \cos(\beta) \cdot \cos(\alpha)$ 
```

onde α é o ângulo azimutal, β é a elevação (limitada a ± 1.5 rad $\approx \pm 86$ para evitar *gimbal lock*), e r é a distância ao foco (Q/E).

A câmera orbital é inicializada a partir da posição XML, convertendo coordenadas cartesianas para esféricas:

$$r = \sqrt{p_x^2 + p_y^2 + p_z^2} \quad (4)$$

$$\alpha = \text{atan2}(p_x, p_z) \quad (5)$$

$$\beta = \arcsin\left(\frac{p_y}{r}\right) \quad (6)$$

2.3.9 Modos de Renderização

Três modos são suportados:

Tecla Modo		Descrição
1	Sólido	Faces preenchidas
2	Wireframe	Apenas arestas
3	Pontos	Apenas vértices

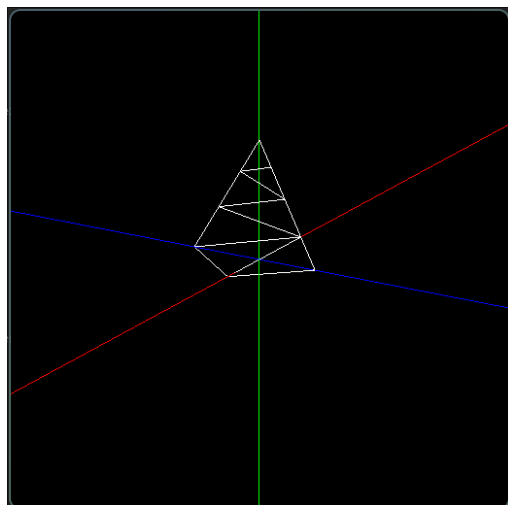
Tabela 1: Modos de renderização disponíveis.

2.3.10 Controlos Adicionais

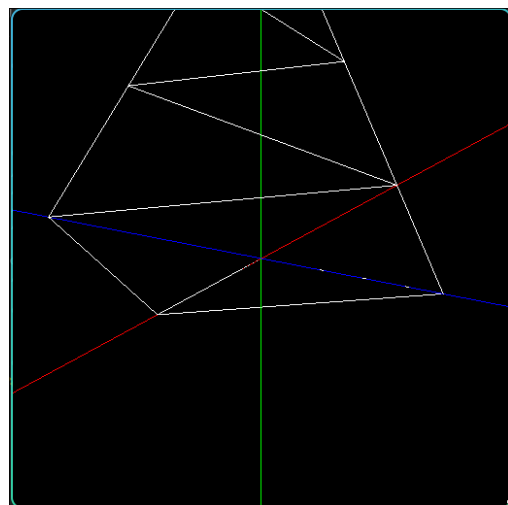
Tecla Ação	
W/S	Rodar câmera verticalmente (elevação)
A/D	Rodar câmera horizontalmente (azimute)
Q/E	Zoom in/out (alterar raio)
C	Alternar câmera fixa/orbital
ESC	Sair da aplicação

Tabela 2: Controlos do engine.

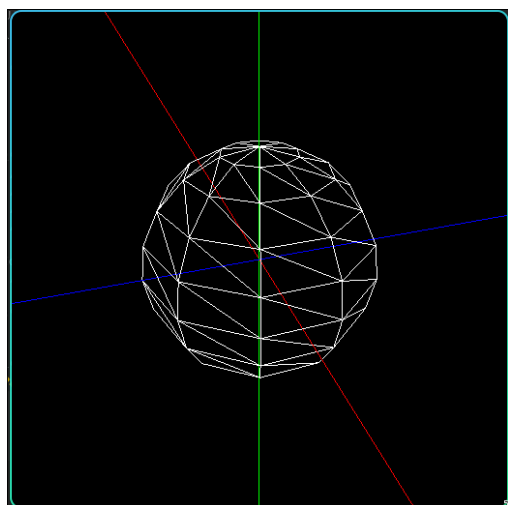
2.4 Resultados obtidos



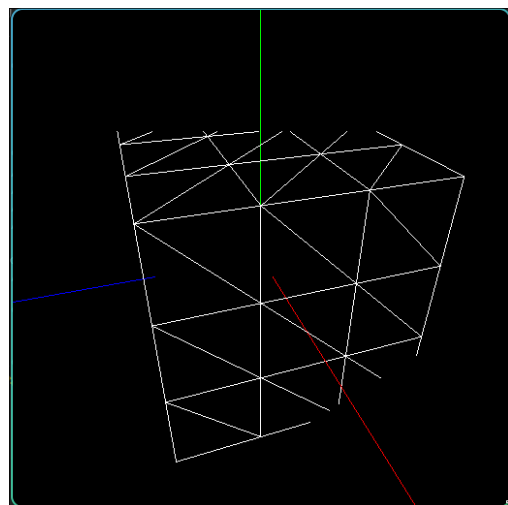
(a) Teste 1_1



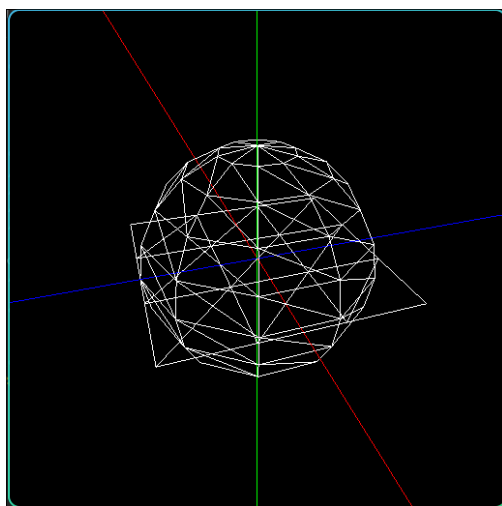
(b) Teste 1_2



(c) Teste 1_3



(d) Teste 1_4



(e) Teste 1_5

Figura 5: Resultados dos testes

2.5 Conclusão

Nesta primeira fase, implementámos corretamente as quatro primitivas que nos foram solicitadas no generator, bem como o engine capaz de ler as configurações em XML e renderizar os modelos com OpenGL/GLUT, tal como praticámos nas aulas. O nosso projeto inclui algumas configurações de câmaras, como a fixa e a orbital, três modos de renderização e os respetivos eixos de referência.

De uma forma geral, o nosso grupo sente que conseguiu realizar, sem grandes dificuldades, o que nos foi proposto para esta primeira fase de entrega. O único detalhe que demorou um pouco mais a ser concretizado foi a confirmação e correção da imagem produzida pelo nosso engine, de modo a que fosse exatamente igual à fornecida na diretoria com as imagens dos resultados esperados. A parte de gerar as figuras decorreu sem problemas. No entanto, garantir que ficassem exatamente iguais às imagens de teste exigiu mais algum tempo.

3 Fase 2: Transformações Geométricas e Hierarquia de Grupos

3.1 Objetivo da Fase 2

O objetivo principal desta fase foi evoluirmos o *engine* de uma renderização plana (lista simples de modelos) para uma renderização hierárquica com transformações geométricas, de acordo com a sintaxe XML proposta no enunciado. Em termos práticos, os requisitos implementados foram:

- suporte às transformações **translate**, **rotate** e **scale**;
- suporte a grupos aninhados (*scene graph*);
- aplicação recursiva de transformações com herança pai-filho;
- preservação da ordem das transformações conforme definida no XML.

3.2 Arquitetura Introduzida

Para suportar a nova organização da cena, introduzimos dois novos tipos de dados no diretório **engine/scene**:

- **Transform**: representa uma transformação individual (tipo + parâmetros);
- **Group**: representa um nó da cena, que contém uma lista de transformações, modelos e grupos filhos.

Esta decisão permitiu separar claramente:

1. a descrição da cena (XML),
2. a estrutura em memória (*scene graph*),
3. a renderização recursiva em OpenGL.

3.2.1 Estrutura Transform :

Cada transformação é armazenada de forma genérica da seguinte forma:

- tipo (**Translate**, **Rotate**, **Scale**),
- componentes **x,y,z**,
- ângulo (que é usado apenas em **Rotate**).

3.2.2 Estrutura Group :

Cada grupo contém:

- **vector<Transform>**, que são as transformações locais do grupo;
- **vector<string>**, são os ficheiros **.3d** desse grupo;
- **vector<Group>**, são os filhos para a renderização recursiva.

Desta maneira, qualquer subgrupo herda automaticamente as transformações acumuladas do seu pai.

3.3 Parsing XML Recursivo

Na Fase 1 o parser carregava apenas uma lista linear de modelos. Na Fase 2 passámos a construir uma árvore de grupos através de uma função recursiva **parseGroup(XMLElement*)**.

3.3.1 Leitura das Transformações As transformações são lidas a partir do nó **<transform>** e inseridas no vetor pela mesma ordem em que surgem no XML. Esta opção é crítica porque transformação geométrica não é comutativa; por exemplo, *translate seguido de rotate* produz resultado diferente de *rotate seguido de translate*.

3.3.2 Leitura dos Modelos Cada nó **<models>** pode conter vários **<model file="...">**. Os nomes dos ficheiros são guardados no grupo corrente.

3.3.3 Leitura de Subgrupos Cada `<group>` pode conter outros `<group>` e a função recursiva cria essa hierarquia em memória. O resultado final é armazenado em `WorldConfig::rootGroup`.

3.4 Renderização Hierárquica

No *engine*, a renderização passou a ser feita por uma função recursiva `renderGroup(const Group&):`

1. `glPushMatrix()` para guardar a matriz atual;
2. aplicar as transformações locais do grupo;
3. desenhar os modelos desse grupo;
4. renderizar recursivamente os filhos;
5. `glPopMatrix()` para restaurar a matriz anterior.

Ou seja, um modelo filho é desenhado no referencial já transformado do pai.

3.4.1 Aplicação das Transformações :

Implementámos uma função auxiliar `applyTransform` com um mapeamento direto para facilitar a implementação posteriormente:

- `Translate` → `glTranslatef(x,y,z)`
- `Rotate` → `glRotatef(angle,x,y,z)`
- `Scale` → `glScalef(x,y,z)`

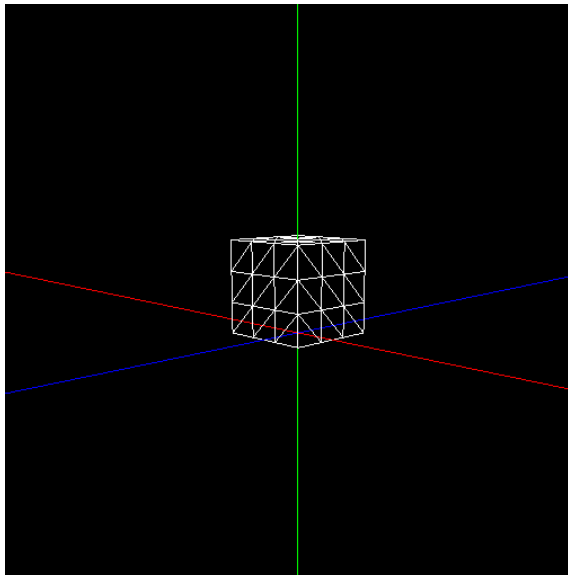
3.5 Validação com Testes da Fase 2

A validação foi feita com os quatro ficheiros disponibilizados para a Fase 2 fornecidos pelos professores:

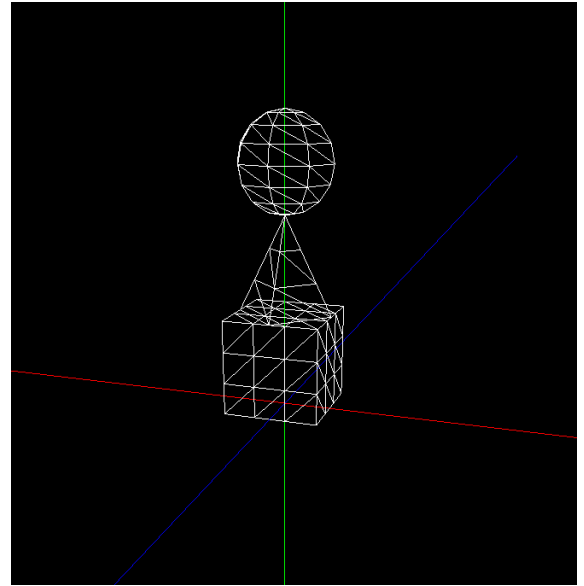
- `test_2_1.xml`, que é uma transformação simples;
- `test_2_2.xml`, uma hierarquia pai-filho-filho;
- `test_2_3.xml`, uma composição de transformações;
- `test_2_4.xml`, com múltiplas caixas em torno do centro e com rotações diferentes.

Durante esta fase surgiram alguns problemas nos testes que envolviam caixas, porque a ordem de geração de algumas faces produzia triângulos diferentes dos esperados nas imagens de referência fornecidas pelos professores.

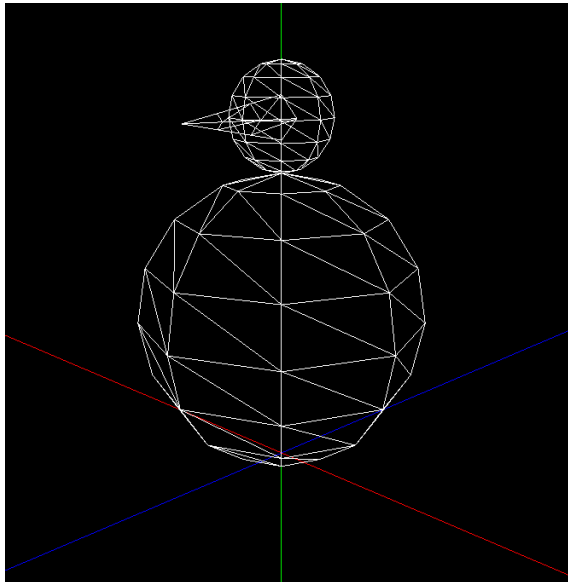
3.6 Resultados Obtidos



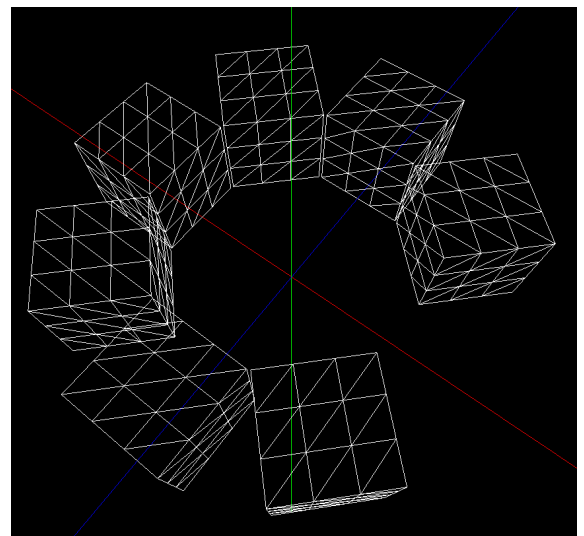
(a) Teste 2_1



(b) Teste 2_2



(c) Teste 2_3



(d) Teste 2_4

Figura 6: Resultados visuais da Fase 2

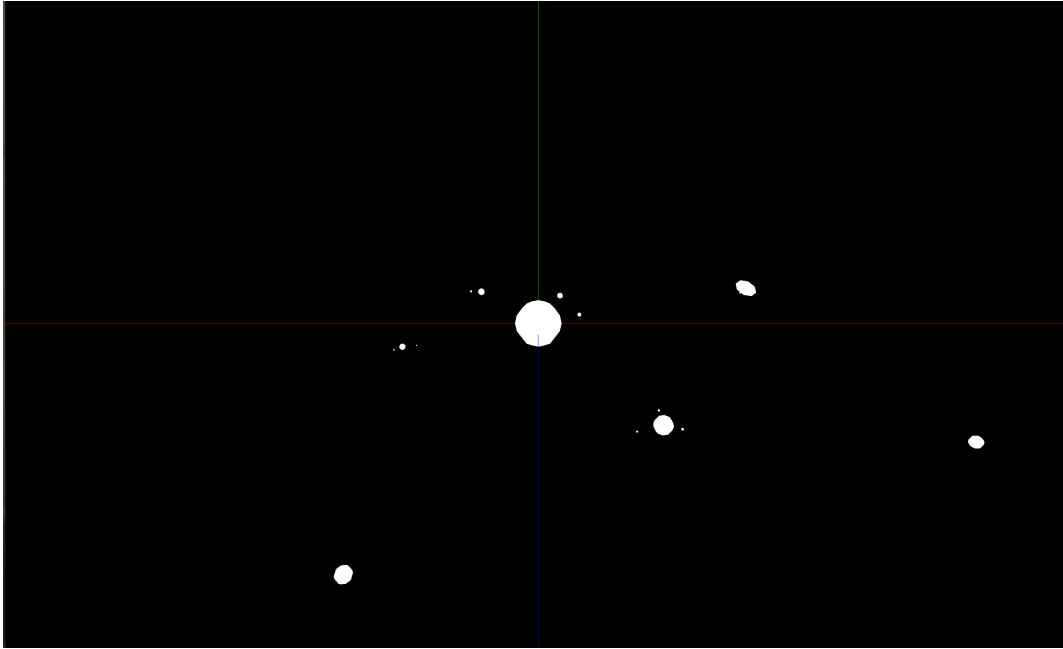


Figura 7: Demo final da Fase 2: sistema solar estático com grupos hierárquicos e transformações geométricas

3.7 Conclusão da Fase 2

A Fase 2 ficou concluída com a introdução de transformações geométricas estáticas e estrutura hierárquica de grupos no *engine*. A principal diferença foi arquitetural (como já tinha sido mencionado no enunciado). Passámos de uma cena plana para uma *scene graph* recursiva, preservando a ordem de transformações e garantindo herança entre níveis.

4 Fase 3: Curvas de Catmull-Rom, Superfícies de Bezier e VBOs

4.1 Objetivo da Fase 3

Nesta fase o objetivo foi acrescentar movimento às cenas através de curvas de Catmull-Rom, introduzir a geração de superfícies de Bezier a partir de ficheiros `.patch` e passar o desenho dos modelos para VBOs. Em termos funcionais, esta fase exigiu três partes principais:

- geração de superfícies de Bezier no *generator*;
- suporte no *engine* para `translate` animado com pontos de controlo e orientação pela tangente da curva;
- carregamento dos vértices para a GPU através de VBOs, evitando o envio imediato dos vértices em cada desenho.

4.2 Geração das Superfícies de Bezier

No *generator*, a implementação passou a ler ficheiros `.patch`, interpretar os índices dos pontos de controlo e calcular os vértices da superfície através das bases de Bernstein. Cada patch cúbico é subdividido numa grelha paramétrica e cada célula dessa grelha é convertida em dois triângulos para formar o ficheiro `.3d` final.

A lógica usada segue a fórmula clássica de Bezier bidimensional:

$$P(u, v) = \sum_{i=0}^3 \sum_{j=0}^3 B_i(u) B_j(v) P_{ij}$$

onde B_i representam os polinómios de Bernstein e P_{ij} os pontos de controlo do patch.

4.2.1 Estratégia de Tesselação A tesselação foi parametrizada por um valor inteiro recebido na linha de comandos. Quanto maior o valor, maior o número de triângulos gerados e mais suave fica a superfície final. Para a validação desta fase foi usado `tess = 10`, através do comando `generator patch teapot.patch 10 bezier_10.3d`.

4.2.2 Formato do Ficheiro Gerado Na Fase 3, o ficheiro gerado manteve o formato das fases anteriores: número total de vértices seguido da lista de pontos (x, y, z) . Esta decisão permitiu reutilizar o leitor de modelos existente e focar a alteração estrutural no uso de VBOs dentro do *engine*.

4.3 Curvas de Catmull-Rom no Engine

No *engine*, a principal alteração no sistema de transformações foi o suporte a uma `translate` temporal, definida por um conjunto de pontos de controlo no XML. Em vez de uma translação fixa, o modelo passa a deslocar-se ao longo de uma curva de Catmull-Rom fechada, calculada a partir desses pontos.

Para cada instante da animação, o *engine* calcula:

- a posição atual na curva;
- a derivada da curva, usada como direção do movimento;
- a matriz de orientação quando a opção `align` está ativa.

4.3.1 Cálculo da Curva O cálculo da curva foi implementado com a fórmula de Catmull-Rom, usando quatro pontos consecutivos para cada segmento. Os índices são calculados com módulo, o que faz com que o último segmento volte ao primeiro ponto e a trajetória fique fechada.

4.3.2 Alinhamento com a Trajetória Quando o atributo `align` é ativado, o objeto não só se desloca ao longo da curva como também roda para acompanhar a direção tangente. Para isso foi construída uma matriz ortonormal a partir da derivada, usando um vetor *up* fixo como referência.

4.3.3 Visualização da Curva Para facilitar a validação visual, a trajetória é desenhada no ecrã em simultâneo com o movimento do objeto. Esta linha ajuda a confirmar se a trajetória e a posição do modelo estão coerentes com os pontos de controlo definidos no XML.

4.4 VBOs

Até à Fase 2, os modelos eram desenhados diretamente a partir dos vetores de vértices em memória. Na Fase 3 foi introduzida uma cache de VBOs, associando cada ficheiro `.3d` a uma estrutura `ModelGPU`.

O processo passou a ser:

1. carregar o ficheiro `.3d` para memória;
2. criar um VBO com `glGenBuffers`;
3. enviar os vértices para a GPU com `glBufferData`;
4. desenhar com `glDrawArrays`.

O carregamento dos VBOs é feito apenas depois de existir contexto OpenGL, evitando erros de inicialização. A cache permite ainda que um mesmo modelo usado várias vezes na cena seja enviado para a GPU apenas uma vez.

4.5 Parsing XML da Fase 3

O parser foi estendido para reconhecer novas variantes de `translate` e `rotate`:

- `translate time="..."` com pontos de controlo;
- atributo `align` para ativar orientação pela trajetória;
- `rotate time="..."` para rotações animadas.

Esta alteração manteve a compatibilidade com as transformações estáticas já usadas na Fase 2. Caso a translação não tenha tempo associado, o comportamento continua a ser uma translação fixa normal.

4.6 Validação com os Testes da Fase 3

A validação foi feita com os testes disponibilizados para a fase 3:

- `test_3_1.xml`, com movimento animado ao longo de uma curva de Catmull-Rom e alinhamento pela trajetória;
- `test_3_2.xml`, com a superfície de Bezier gerada a partir do ficheiro `teapot.patch`.

4.7 Resultados Obtidos

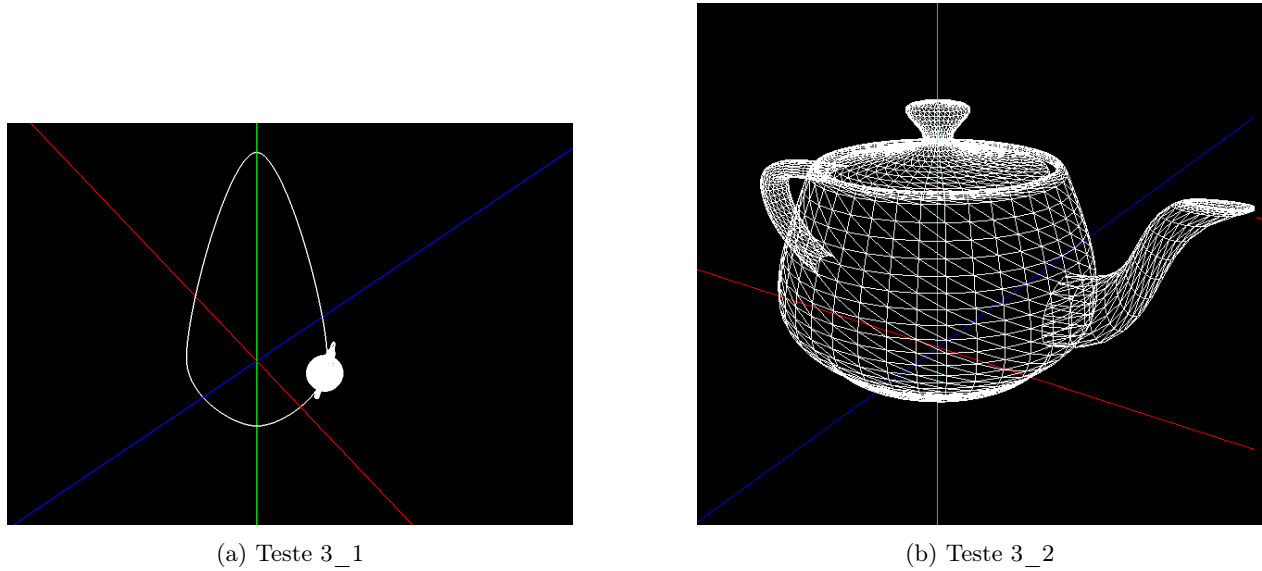


Figura 8: Resultados visuais da Fase 3

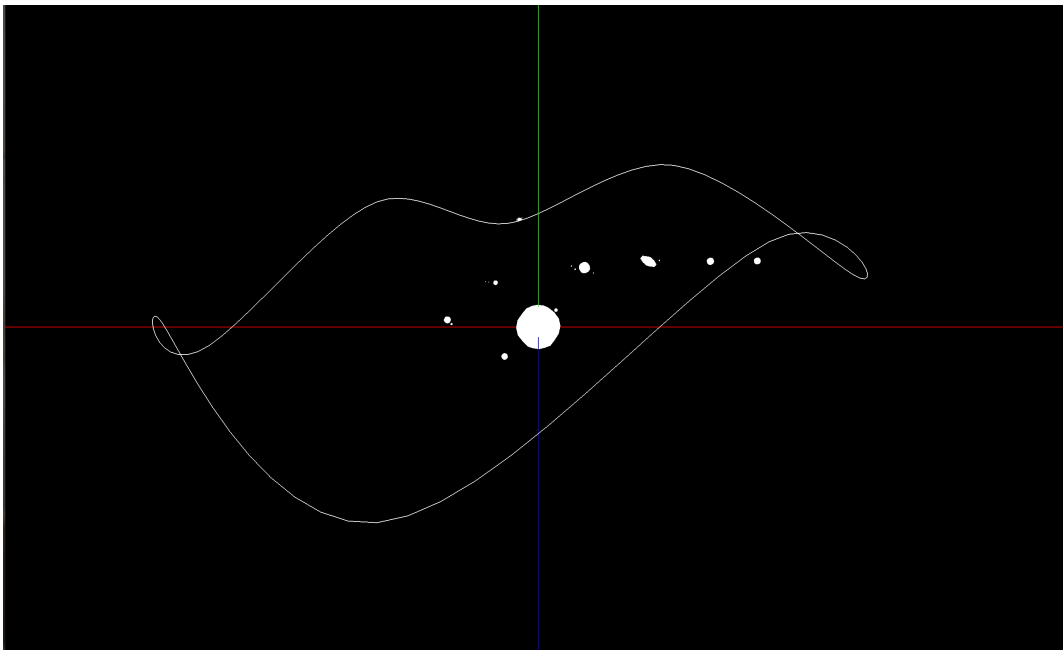


Figura 9: Demo final da Fase 3: sistema solar animado e objeto Bezier a deslocar-se numa curva de Catmull-Rom

4.8 Conclusão da Fase 3

A Fase 3 acrescentou animação temporal ao *engine*, geração de superfícies de Bezier no *generator* e utilização de VBOs para desenhar os modelos. Estas alterações prepararam diretamente a Fase 4: a cache

de modelos e o envio para a GPU foram depois estendidos para incluir também normais e coordenadas de textura.

Durante o desenvolvimento desta fase, uma das principais dificuldades foi garantir que os novos campos das transformações, como os pontos de controlo e o atributo `align`, se integravam bem com o parser e com a estrutura hierárquica já existente. Também foi necessário garantir que os VBOs eram criados apenas após a criação do contexto OpenGL.

5 Fase 4: Normais, Iluminação, Materiais e Texturas

5.1 Objetivo da Fase 4

Na Fase 4 o objetivo foi evoluir o motor gráfico para suportar uma renderização mais próxima do modelo de iluminação do OpenGL. Até à fase anterior, os modelos eram desenhados essencialmente a partir das suas posições geométricas. Nesta fase passamos a guardar e usar informação adicional por vértice:

- normais, necessárias para o cálculo da iluminação;
- coordenadas de textura, necessárias para aplicar imagens sobre as superfícies;
- materiais por modelo, definidos no XML através de componentes *diffuse*, *ambient*, *specular*, *emissive* e *shininess*;
- luzes globais da cena, incluindo luzes pontuais, direcionais e *spotlights*.

Além disso, a renderização foi adaptada para usar VBOs também para normais e coordenadas de textura, mantendo a estratégia iniciada na fase anterior de carregar os modelos para a GPU e evitar leituras repetidas dos ficheiros durante o desenho.

5.2 Novo Formato dos Ficheiros .3d

Para suportar iluminação e texturas, o formato dos ficheiros gerados pelo *generator* foi estendido. Em vez de guardar apenas a lista de vértices, cada ficheiro passa a ter três blocos de dados alinhados:

1. número total de vértices;
2. lista dos vértices (x, y, z) ;
3. lista das normais (n_x, n_y, n_z) ;
4. lista das coordenadas de textura (s, t) .

A posição i de cada bloco corresponde ao mesmo vértice. Ou seja, o vértice i , a normal i e a coordenada de textura i são tratados como um único conjunto de dados durante a renderização.

Para organizar esta informação foi introduzida a estrutura `geometryPointData`, composta por:

- `vector<Point3D> vertices;`
- `vector<Point3D> normals;`
- `vector<Point2D> texCoords.`

O `FileWriter` foi também adaptado para validar que os três vetores têm o mesmo tamanho antes de escrever o ficheiro. Esta validação é importante porque o *engine* assume que os dados estão alinhados por índice.

5.3 Normais e Coordenadas de Textura no Generator

Todas as primitivas foram atualizadas para produzir, além dos vértices, uma normal e uma coordenada de textura para cada ponto emitido.

5.3.1 Plano

No plano, a normal é constante e aponta para cima:

$$n = (0, 1, 0)$$

As coordenadas de textura são calculadas diretamente a partir da posição da célula na grelha. Para uma subdivisão (i, j) , os valores s e t são normalizados pelo número total de divisões, ficando no intervalo $[0, 1]$. Esta abordagem permite aplicar uma textura sobre todo o plano de forma contínua.

5.3.2 Caixa

Na caixa, cada face tem uma normal constante, alinhada com o eixo perpendicular a essa face:

- frente e trás: $(0, 0, 1)$ e $(0, 0, -1)$;
- direita e esquerda: $(1, 0, 0)$ e $(-1, 0, 0)$;
- cima e baixo: $(0, 1, 0)$ e $(0, -1, 0)$.

As coordenadas de textura são calculadas localmente por face, mapeando as coordenadas da face para o intervalo $[0, 1]$. Esta decisão mantém o mapeamento simples e previsível, repetindo a imagem de forma independente em cada face da caixa.

5.3.3 Esfera

Na esfera, como o centro está na origem, a normal de cada vértice é obtida normalizando o vetor posição:

$$n = \frac{p}{r}$$

As coordenadas de textura são calculadas a partir dos ângulos esféricos:

$$s = \frac{\phi}{2\pi} \quad t = 1 - \frac{\theta}{\pi}$$

A inversão em t foi usada para alinhar a orientação da imagem com a visualização esperada no OpenGL.

5.3.4 Cone

No cone, a base usa a normal constante $(0, -1, 0)$. As suas coordenadas de textura são calculadas a partir de uma projeção polar, usando o ângulo do ponto e a distância ao centro da base.

Na superfície lateral, a normal é calculada a partir da posição do ponto e da inclinação do cone:

$$n = \text{normalize}\left(x, \frac{R}{h}, z\right)$$

Esta aproximação permite obter uma iluminação suave ao longo da lateral. As coordenadas de textura laterais usam a fatia como coordenada s e a camada como coordenada t .

5.3.5 Superfícies de Bezier

As superfícies de Bezier também passaram a emitir normais e coordenadas de textura. Para as coordenadas de textura, usamos diretamente os parâmetros u e v da tesselação da superfície.

Para as normais, foram consideradas as derivadas parciais da superfície, embora na versão entregue a normal emitida seja calculada por triângulo através do produto vetorial das arestas. Esta opção produziu resultados visualmente alinhados com os testes usados na validação.

5.4 Carregamento no Engine e Uso de VBOs

O *engine* foi atualizado para ler o novo formato dos ficheiros `.3d`. A estrutura `Model` passou a guardar três vetores: vértices, normais e coordenadas de textura.

Para manter compatibilidade com ficheiros antigos, o carregamento aceita a ausência de normais ou coordenadas de textura. Se as normais não existirem, o modelo é carregado apenas com vértices. Se existirem normais mas não existirem coordenadas de textura, o modelo continua a poder ser desenhado com iluminação, mas sem textura.

No lado da GPU, foi introduzida a estrutura `ModelGPU`, que mantém:

- `vboVertices`;
- `vboNormals`;
- `vboTexCoords`;
- número de vértices;
- flags para indicar se existem normais e coordenadas de textura.

O `gpuCache` associa cada caminho de ficheiro ao respetivo conjunto de VBOs. Assim, cada modelo é carregado e enviado para a GPU apenas uma vez. Durante o desenho, o *engine* ativa os estados `GL_VERTEX_ARRAY`, `GL_NORMAL_ARRAY` e `GL_TEXTURE_COORD_ARRAY` conforme os dados disponíveis, e desenha com `glDrawArrays`.

5.5 Materiais no XML

O parser foi estendido para reconhecer, dentro de cada `model`, a secção `color`. Cada componente é lida em valores de 0 a 255 e convertida para o intervalo $[0, 1]$, que é o formato esperado pelas chamadas de material do OpenGL.

As propriedades suportadas são:

- `diffuse`, para a cor principal do objeto;
- `ambient`, para a componente ambiente;
- `specular`, para reflexos especulares;
- `emissive`, para emissão própria;
- `shininess`, para controlar a concentração do brilho especular.

Durante a renderização, antes de desenhar cada modelo, o *engine* chama `applyMaterial`, que traduz estes valores para chamadas `glMaterialfv` e `glMaterialf`.

5.6 Luzes

Foi introduzida no XML uma secção `lights`. O parser suporta três tipos de luz:

- `point`, definida por posição;
- `directional`, definida por direção;
- `spot`, definida por posição, direção e ângulo de corte.

No OpenGL, estas luzes são aplicadas através de `GL_LIGHT0`, `GL_LIGHT1`, etc., até ao limite de 8 luzes do pipeline fixo. Quando existem luzes no XML, o *engine* ativa `GL_LIGHTING`. Quando não existem, a iluminação é desativada para manter compatibilidade com cenas mais simples.

Também foram ativadas opções para melhorar a robustez da iluminação:

- `GL_NORMALIZE`, para garantir normais normalizadas antes do cálculo de luz;
- `GL_RESCALE_NORMAL`, relevante quando existem escalas aplicadas aos modelos;
- luz ambiente global através de `glLightModelfv`.

5.7 Texturas com DevIL

Para carregar imagens, foi usada a biblioteca DevIL. A inicialização é feita após a criação do contexto OpenGL, garantindo que as texturas podem ser corretamente enviadas para a GPU.

O fluxo da implementação é o seguinte:

1. carregar a imagem com DevIL;
2. converter a imagem para RGBA;
3. criar uma textura OpenGL com `glGenTextures`;
4. configurar repetição com `GL_REPEAT`;
5. configurar filtros lineares;
6. enviar os dados com `glTexImage2D`.

Tal como nos modelos, as texturas também usam uma cache, neste caso `texturaCache`, indexada pelo caminho do ficheiro. Desta forma, a mesma textura não é carregada várias vezes.

Ao desenhar um modelo, a função `applyTextura` verifica se o XML define uma textura e se o modelo tem coordenadas de textura válidas. Se ambas as condições forem verdadeiras, ativa `GL_TEXTURE_2D`, faz *bind* da textura e usa `GL_MODULATE`, permitindo combinar a textura com o material definido para o modelo.

5.8 Validação com os Testes da Fase 4

A validação foi feita com os testes disponibilizados para a fase 4:

- `test_4_1.xml`, com uma luz direcional e material difuso;
- `test_4_2.xml`, com componente emissiva;
- `test_4_3.xml`, com duas luzes pontuais e vários objetos;
- `test_4_4.xml`, com uma luz pontual próxima da cena;
- `test_4_5.xml`, com uma luz do tipo *spot*;
- `test_4_6.xml`, com texturas aplicadas ao plano, cone, esfera, superfície de Bezier e caixa.

5.9 Resultados Obtidos

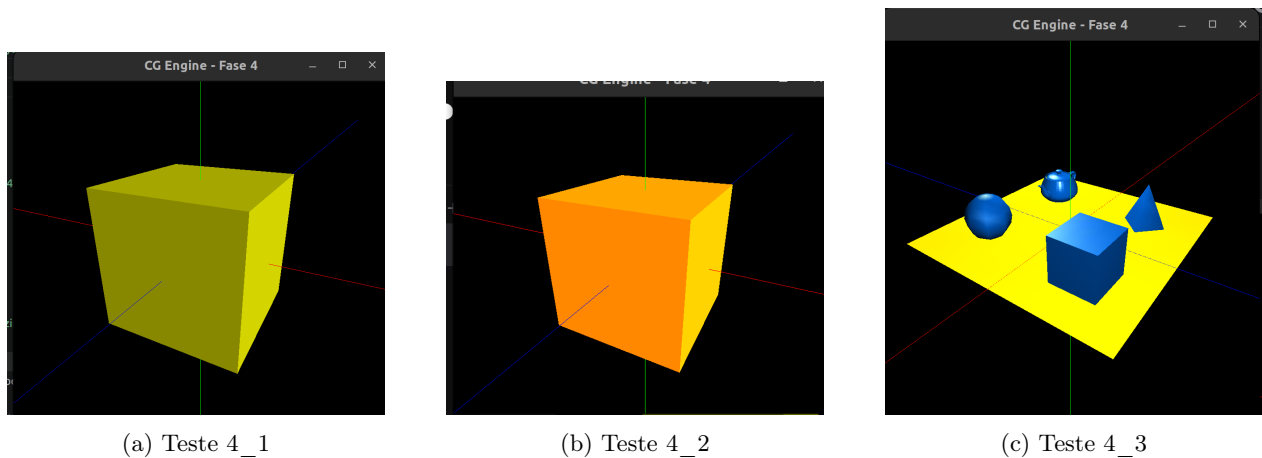
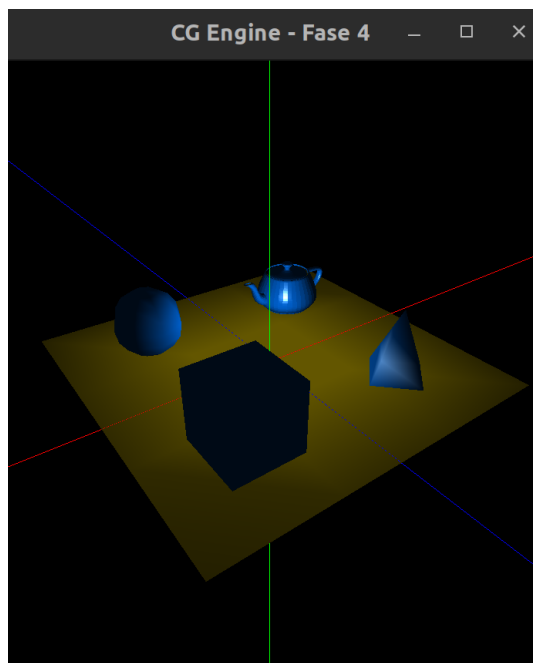
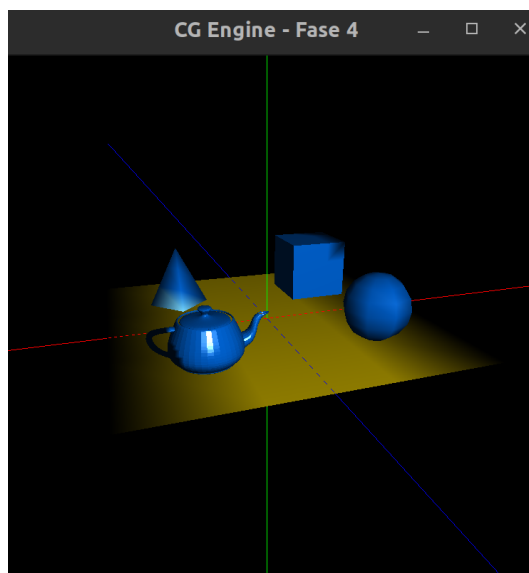


Figura 10: Resultados visuais dos testes 4_1, 4_2 e 4_3

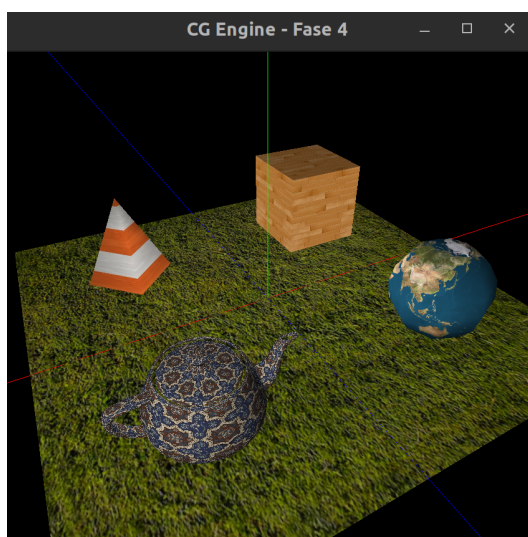


(a) Teste 4_4



(b) Teste 4_5

Figura 11: Resultados visuais dos testes 4_4 e 4_5



(a) Teste 4_6

Figura 12: Resultados visuais com texturas

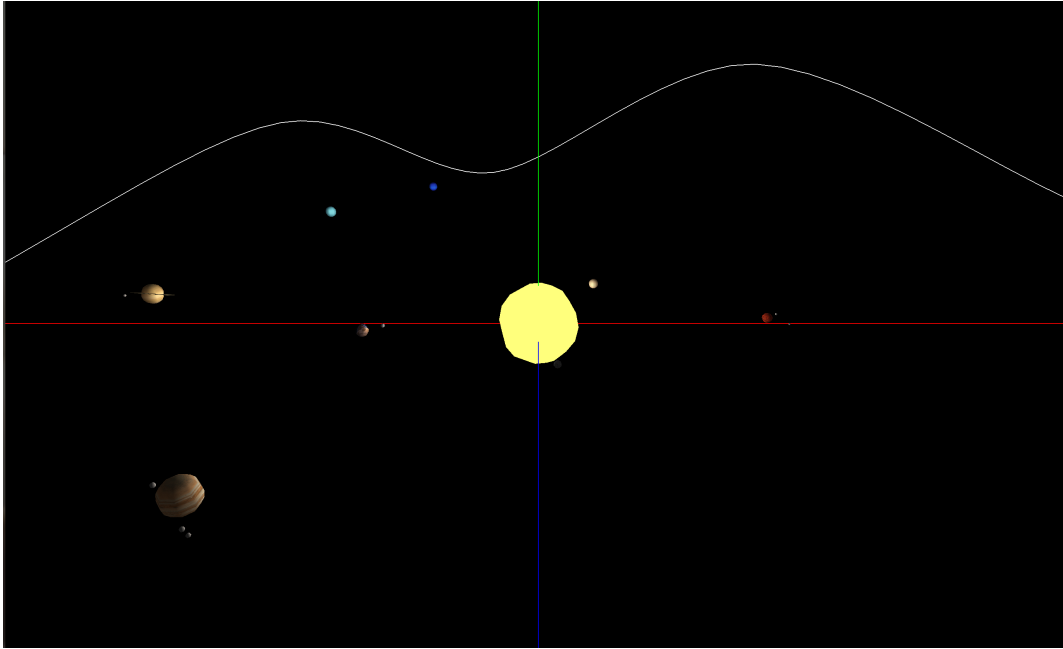


Figura 13: Demo final da Fase 4: sistema solar animado com materiais, texturas e iluminação

5.10 Conclusão da Fase 4

A Fase 4 completou a transição do motor para uma renderização com iluminação e texturas. A alteração principal foi a passagem de modelos puramente geométricos para modelos com atributos por vértice, o que obrigou a alterar o *generator*, o formato dos ficheiros *.3d*, o carregamento no *engine* e o envio dos dados para a GPU.

As principais dificuldades estiveram relacionadas com a coerência entre os três vetores de dados por vértice, a orientação correta das normais e o mapeamento das texturas em cada primitiva. Também foi necessário garantir que o carregamento de imagens com DevIL acontecia apenas depois de existir contexto OpenGL, tal como já tinha acontecido com os VBOs nas fases anteriores.

No final, o motor passou a suportar as funcionalidades essenciais da fase: luzes, materiais, texturas, normais e coordenadas de textura, mantendo a estrutura hierárquica e as animações implementadas anteriormente.

6 Conclusões

O trabalho permitiu construir progressivamente um motor gráfico 3D dividido entre um *generator*, responsável pela criação dos modelos, e um *engine*, responsável pela leitura da cena e pela renderização. Ao longo das quatro fases, a solução evoluiu de uma visualização simples de primitivas para uma estrutura hierárquica com transformações, animações, superfícies de Bezier, VBOs, iluminação, materiais e texturas.

A separação entre geração de geometria e renderização revelou-se importante, porque permitiu acrescentar novos atributos aos modelos sem alterar a lógica principal de definição das cenas em XML. A introdução de estruturas como **Group**, **Transform**, **ModelInfo** e **geometryPointData** ajudou a manter o código organizado à medida que os requisitos aumentaram.

Na fase final, o maior impacto esteve na passagem de modelos apenas com vértices para modelos com normais e coordenadas de textura. Esta alteração tornou possível usar iluminação e texturas de forma coerente, mas também exigiu maior cuidado na geração dos dados, no carregamento dos ficheiros e na ativação correta dos estados do OpenGL.

De forma geral, consideramos que os objetivos principais do trabalho foram cumpridos. O motor suporta as funcionalidades pedidas nas quatro fases e os testes fornecidos foram usados para validar o comportamento visual das implementações.

Referências

1. Não foram utilizadas referências bibliográficas externas. O trabalho foi desenvolvido com base nos slides teóricos, práticos e nos guiões lecionados na unidade curricular.